

Assignment #2: Exploring the Relationship Between DevOps & Agile

Task 1: Understanding Agile Methodology

Overview, Key Principles, and Practices

Agile is a way of developing software where the team builds the product little by little instead of trying to finish everything in one long process. Instead of planning the whole project from start to finish before writing any code, an Agile team works in short cycles, gets feedback from the customer often, and adjusts what they are building based on that feedback. This is very different from how software used to be built, where a team would spend months planning and documenting before any actual coding started.

The main ideas behind Agile come from the Agile Manifesto, which values things like working software over long documentation, responding to change instead of sticking strictly to a plan, and having the development team talk directly with the customer instead of relying only on contracts and requirement documents. In practice, this means the team plans in small chunks, checks in with each other regularly, and treats changes in requirements as something normal instead of something to avoid (Atlassian, n.d.).

How Agile Solves the Problems of Traditional Development

The traditional way of developing software, usually called the waterfall model, goes through separate stages one after another: first requirements, then design, then coding, then testing, and finally deployment. The problem with this is that the customer usually does not see the actual product until the very end, so if something is wrong or the requirements changed along the way, it is expensive and time consuming to fix. Agile solves this by breaking the whole project into small pieces that get built, tested, and shown to the customer within a short period of time, sometimes just a few weeks. Because feedback comes in early and often, the team can catch problems and misunderstandings

much sooner instead of finding out at the end that the product does not match what the client actually needed.

Two Popular Agile Frameworks

Extreme Programming (XP). Extreme Programming, or XP, is one of the earliest Agile frameworks. It was created by Kent Beck in 1996 and is different from Scrum or Kanban because it focuses more on the actual coding practices of the developers rather than on managing the project as a whole (Scribd, n.d.). XP pushes basic good programming habits to a very high, or "extreme," level. For example, code reviews are replaced with pair programming, where two developers work together at the same computer the whole time, one writing the code and the other reviewing it as it is written. XP also relies heavily on test driven development, meaning tests are written before the actual code, and on continuous integration, where code changes are merged and tested constantly instead of once in a while. Other core practices include collective code ownership, where any developer can change any part of the codebase, and a sustainable pace of work so the team does not burn out from constant overtime. XP was built around five core values: communication, simplicity, feedback, courage, and respect (TechnologyAdvice, 2023).

Lean Software Development. Lean Software Development takes its ideas from lean manufacturing, most famously from the Toyota Production System, and applies them to building software. The main goal of Lean is to remove anything in the process that does not add value to the customer, which is usually called "waste." This waste can be things like unnecessary features, waiting time between tasks, defects that were not caught early, or too much documentation that nobody actually uses. Lean is guided by seven principles: eliminate waste, amplify learning, decide as late as possible, deliver as fast as possible, empower the team, build quality into the product from the start, and see the whole process rather than just individual parts of it (GeeksforGeeks, 2025). Unlike XP, which is mostly about how code is written, Lean is more about how the whole workflow is organized, which is why it is often combined with other frameworks such as Kanban to help visualize and manage the flow of work.

Task 2: Integrating Agile and DevOps

How Agile and DevOps Fit Together

Even though Agile and DevOps are sometimes talked about separately, they are really meant to complement each other rather than compete. Agile is mostly about how a team plans and builds the product step by step, while DevOps is about how that product actually gets delivered to real users in a safe and repeatable way. In a team that uses both, Agile frameworks like XP or Lean guide how the team organizes and prioritizes their work, while DevOps practices such as continuous integration, automated testing, and continuous delivery make sure that whatever the team finishes can be released quickly instead of sitting around waiting for a big manual release later. Basically, Agile decides what gets built and in what order, and DevOps makes sure it can actually reach the users without a slow, risky release process getting in the way (Red Hat, n.d.).

Benefits of Combining Agile and DevOps

- Software gets released faster because small pieces of work can be tested and deployed automatically instead of waiting for one huge release.
- Bugs are usually caught earlier since automated testing runs constantly instead of only at the end of a long development cycle.
- Developers and operations teams communicate more, since DevOps removes the wall that used to exist between the people who write the code and the people who run it in production.
- Feedback loops become shorter, so if something goes wrong, the team can fix it quickly instead of dealing with a huge, risky release.
- The whole team stays closer to what the business actually needs because they are releasing and getting feedback constantly instead of only at the end of the project.

Example: Aligning Sprints with a CI/CD Pipeline

To make this more concrete, imagine an Agile team that works in two week sprints. At the start of the sprint, the team agrees on which tasks they will finish. As each developer finishes a task, they push their code to a shared repository, and this automatically starts the CI/CD pipeline. During continuous integration, the system builds the app and runs automated tests right away, so if something is broken, the team finds out within minutes instead of waiting until the sprint review. Once the code passes these tests, the continuous delivery part of the pipeline can move it to a staging environment for more testing, and depending on how confident the team is, it can even go straight to production. Because this whole pipeline runs every time someone pushes code, by the time the sprint review happens, the features being demonstrated are often already running in an environment that looks like production. This is really the core of how Agile and DevOps work together. Agile gives the team the short cycles and constant feedback, and DevOps makes sure that what gets built during those cycles does not just sit there but actually reaches the user quickly and safely (Sharief, n.d.).

References

- Atlassian. (n.d.). *Agile project management*. Retrieved August 15, 2026, from <https://www.atlassian.com/agile/project-management>
- GeeksforGeeks. (2025, July 23). *Lean software development (LSD)*. <https://www.geeksforgeeks.org/software-engineering/lean-software-development-lsd/>
- Lombardi, R. (2020, January 24). *How an Agile software process is changing the world for non-tech companies* [Video]. TED Conferences. https://www.ted.com/talks/roula_lombardi_how_an_agile_software_process_is_changing_the_world_for_non_tech_companies
- Red Hat. (n.d.). *Why choose Red Hat for agile integration?* Retrieved August 15, 2026, from <https://www.redhat.com/en/topics/integration/why-choose-red-hat-integration>
- Sharief, I. (n.d.). *How to integrate DevOps CI/CD pipelines into Scrum sprint cycles*. Medium. Retrieved August 15, 2026, from <https://medium.com/@kmv491712/how-to-integrate-devops-ci-cd-pipelines-into-scrum-sprint-cycles-b028e88c1323>
- TechnologyAdvice. (2023, September 21). *What is extreme programming (XP)?* <https://technologyadvice.com/blog/project-management/what-is-extreme-programming/>